计算机系统基础实验报告

Lab2：BombLab

从汇编行为到参数化求解器

课程名称	计算机系统基础
姓名	徐子钰
学号	25800440037
完成日期	2026 年 10 月

摘要

本实验以林夏等人在一座停摆的城市中沿着旧记录排查故障、恢复系统，并逐步了解往事的故事为背景展开。在这一叙事线索下，本实验要求通过反汇编和动态调试分析 Bomb 程序，依次完成六个普通关卡及 Honor 关。由于 challenge 会随 seed 改变，我先从 phase 函数确认输入个数和比较位置，再沿函数调用、跳转表、递归或数据结构逐步还原计算关系，最后把它们写成以 challenge 为参数的 C 代码。所有运算都按 32 位无符号整数处理，并使用公开测试以及额外的 seed 43 进行复核。最终六个普通关卡公开测试为 60/60，Honor 关公开测试为 10/10，seed 43 的完整流程也能一次通过。

关键词：反汇编；GDB；x86-64；无符号整数；递归；跳转表；链表

目录

摘要 2
目录 2
一、实验任务与分析方法 4
1.1 任务理解 4
1.2 统一的阅读顺序 4
1.3 验证策略 4
二、Phase 1：拆分 challenge 并组合三个步骤 5
2.1 汇编线索 5
2.2 还原关系与实现 5
三、Phase 2：从循环寄存器还原数列 7
3.1 首项与循环 7
3.2 数列公式与实现 8
四、Phase 3：识别跳转表 9
4.1 selector 与跳转表 9
4.2 switch 实现与验证 10
五、Phase 4：递归二分搜索与回溯 11

5.1 递归结构 11
5.2 用显式栈模拟回溯 12
六、Phase 5：从内存偏移恢复链表 14
6.1 节点布局 14
6.2 seed 43 的链路核对 15
七、Phase 6：函数指针表与两段处理流水线 16
7.1 两次间接调用 16
7.2 实现与验证 17
八、Honor：识别门控字符串并恢复隐藏计算 19
8.1 secret_gate 的输入布局 19
8.2 三个字段的计算 19
九、测试结果与完整性检查 21
9.1 测试汇总 21
9.2 对结果的理解 22
十、收获与思考 22
十一、工具使用说明与参考资料 23
11.1 工具使用说明 23
11.2 参考资料 23

一、实验任务与分析方法

1.1 任务理解

BombLab 给出的 bomb 会根据 seed 生成一个 64 位 challenge，再把我输入的若干个无符号十进制整数交给对应关卡检查。实验需要完成 defuser.c：它收到同一个 challenge 后，必须算出 bomb 正在等待的输入。后续分析按照“寻找证据、还原关系、编写实现、运行验证”的顺序展开。

开始正式关卡前，我先在 tutor 中用 phase 0 熟悉了 GDB 的断点、寄存器和单步执行。这个练习让我明确了两件事：反汇编里的 shr、xor、lea 是处理器执行的指令，不能直接当成 GDB 命令输入；GDB 中的 ni、si、x、p、disassemble 才是观察程序的命令。正式分析时，我先用 objdump 查看整体结构，再使用 GDB 在关键位置核对寄存器和内存。

1.2 统一的阅读顺序

每一关我都先看 phase_0x 的外层函数。valid_u32_fields 和 sscanf 能确定需要几个整数，最后的 cmp 则告诉我计算结果与输入数组的哪一项比较。随后再进入辅助函数，把寄存器的来源写成有意义的变量名。按照 System V AMD64 调用约定，前几个整数或指针参数依次放在%rdi、%rsi、%rdx、%rcx、%r8、%r9 中，32 位子寄存器%eax、%edi 等只保留低 32 位。这样可以较快地区分 challenge、循环计数和临时结果。

最终答案都用 uint32_t 保存，它表示一个 32 位无符号整数，取值范围是 0 到 4294967295。如果加法或乘法的结果超过上限，或者减法算出了负数，程序就会像计数器转完一圈一样回到这个范围内，只保留最低 32 位。处理器把结果写入 32 位寄存器时也是同样的效果。代码中的 UINT32_C 和 UINT64_C 用于告诉编译器，相应常量要分别按 32 位或 64 位无符号整数处理，避免不同类型一起运算时出现意外结果。

$$r = x \bmod 2^{32}, \quad 0 \le r < 2^{32}$$

1.3 验证策略

公开脚本覆盖了一组固定 seed，但实验还要求每关使用至少一个公开列表之外的 seed。这里统一选用 seed 43。单关验证时，我使用管道符|将 defuser 生成的答案直接传

递给 bomb，并保持两侧的 seed 和 phase 参数一致。全部完成后再运行公开测试和包含 Honor 的完整流程。

二、Phase 1：拆分 challenge 并组合三个步骤

2.1 汇编线索

外层 phase_01 只读取一个无符号整数，随后把 challenge 交给 p1_compute。p1_compute 先保存 challenge 的低 32 位，再分别右移 16 位和 32 位，说明三个辅助函数使用的是 challenge 的重叠切片。三个小函数中最关键的指令如下。

关键汇编

```
# p1_step1
lea    (%rdi,%rdi,2),%eax
add    %esi,%eax
xor    $0xf5a6,%eax

# p1_step0
xor    $0x572e,%esi
lea    (%rsi,%rdi,4),%eax

# p1_step2
sub    %esi,%edi
lea    0x1565(%rdi,%rdi,2),%eax
```

lea 在这里没有访问内存，只做乘加运算。再结合 p1_compute 的传参顺序，可以依次写出 step1、step0 和最终 step2。

2.2 还原关系与实现

$$\text{high} = C[63:32], \quad \text{middle} = C[47:16], \quad \text{low} = C[31:0]$$

$$\text{step1} = (3 \cdot \text{high} + \text{middle}) \oplus \text{0xf5a6}$$

$$\text{step0} = (\text{high} \oplus \text{0x572e}) + 4 \cdot \text{low}$$

$$\text{answer} = 3 \cdot (\text{step0} - \text{step1}) + \text{0x1565}$$

对应的 C 代码

```
const uint32_t high = (uint32_t)(challenge >> 32);

const uint32_t middle = (uint32_t)(challenge >> 16);

const uint32_t low = (uint32_t)challenge;

const uint32_t step1 = (high * UINT32_C(3) + middle) ^ UINT32_C(0xf5a6);

const uint32_t step0 = (high ^ UINT32_C(0x572e)) + low * UINT32_C(4);

values[0] = (step0 - step1) * UINT32_C(3) + UINT32_C(0x1565);
```

seed 43 时，challenge 为 0x9cde98852e60034b。代入后得到 step1=1864368713、step0=1449057495，32 位减法发生回绕，最终答案为 3049039119。



图 1 Phase 1 使用额外 seed 43 的运行结果

三、Phase 2：从循环寄存器还原数列

3.1 首项与循环

Phase 2 需要六个整数。外层函数的第一段先生成首项，后面是一段运行五次的循环。首项附近的 shr $0x12 表示右移 18 位，and $0x3f 说明只保留低 6 位，因而该字段的范围是 0～63；再加 12 得到第一个数。

关键汇编

```
shr    $0x12,%rax

and    $0x3f,%eax

add    $0xc,%eax
```

```
mov    $0x13,%r8d       # 初始移位量 19
mov    $0x7,%esi        # 初始异或量 7
...
shr    %cl,%rax
and    $0x3f,%eax
add    %edi,%eax        # 加上前一项
xor    %esi,%eax
add    $0x7,%esi
add    $0x4,%rdx        # 下一次移位量增加 4
```

循环里移位量从 19 开始，每轮增加 4；异或量从 7 开始，每轮增加 7。当前结果还要与上一项相加，因此后五项会在前一项的基础上依次生成。

3.2 数列公式与实现

$$a_0 = ((C \gg 18)\,\&\,0\text{x}3\text{f}) + 12$$

$$a_i = [a_{i-1} + ((C \gg (19 + 4(i-1)))\,\&\,0\text{x}3\text{f})] \oplus (7i), \quad i = 1,2,3,4,5$$

对应的 C 代码

```
values[0] = (uint32_t)((challenge >> 18) & UINT64_C(0x3f)) + UINT32_C(12);
for (size_t i = 1; i < 6; ++i) {
    const unsigned int shift = 19U + 4U * (unsigned int)(i - 1);
    const uint32_t part = (uint32_t)((challenge >> shift) & UINT64_C(0x3f));
    values[i] = (values[i - 1] + part) ^ (UINT32_C(7) * (uint32_t)i);
}
```

seed 43 的 challenge 为 0x6ec624aa85685867，六个答案依次为 38、84、80、117、150、136。



图 2 Phase 2 使用额外 seed 43 的运行结果

四、Phase 3：识别跳转表

4.1 selector 与跳转表

Phase 3 读取两个整数。第一个整数先与 7 比较，随后用于间接跳转，因此它是 0～7 的 selector。challenge 的最低 3 位天然落在这个范围内，所以可以推出第一个答案就是 challenge&7。另一个公共输入 x 来自 challenge 的两个 32 位部分。

$$\text{selector} = C\&7, x = (\texttt{uint32_t})(C \gg 3) \oplus (\texttt{uint32_t})(C \gg 32)$$

关键汇编

```
and    $0x7,%eax

cmp    $0x7,%eax

ja     <failure>

notrack jmp *0x405820(,%rax,8)
```

这里真正的对应关系由 0x405820 开始的地址表决定，函数名中的编号并不等于表下标。我逐项读取八个表项，再分别反汇编对应的 case 函数，得到下面的映射。

selector	实际分支	输出关系
0	p3_case_7	$3x + 0x6a69$
1	p3_case_4	$4x \oplus 0xebd9$
2	p3_case_6	$(x \oplus 0x726a) + (x \gg 2)$
3	p3_case_5	$5x + 0x3b3ad$
4	p3_case_1	$5x + 0x1d4e$
5	p3_case_3	$(x \oplus 0x607) + (x \gg 1)$
6	p3_case_2	$2x \oplus 0xe506$
7	p3_case_0	$5x + 0x16445$

4.2 switch 实现与验证

映射确定后，用 switch 重写最直观，也方便逐项与跳转表核对。下面保留 seed 43 命中的 selector 7 分支；其余七个分支按照上表展开。

核心 C 代码

```
const uint32_t x = (uint32_t)(challenge >> 3) ^ (uint32_t)(challenge >> 32);
values[0] = (uint32_t)(challenge & UINT64_C(7));
switch (values[0]) {
    /* 其余分支按跳转表逐项实现 */
    case 7:
        values[1] = x * UINT32_C(5) + UINT32_C(0x16445);
```

```
        break;
}
```

seed 43 时，challenge 为 0xd49ffe504bfdab7f，selector=7，x=3722464063，第二个答案在 32 位运算后为 1432542336。因此完整输入为“7 1432542336”。

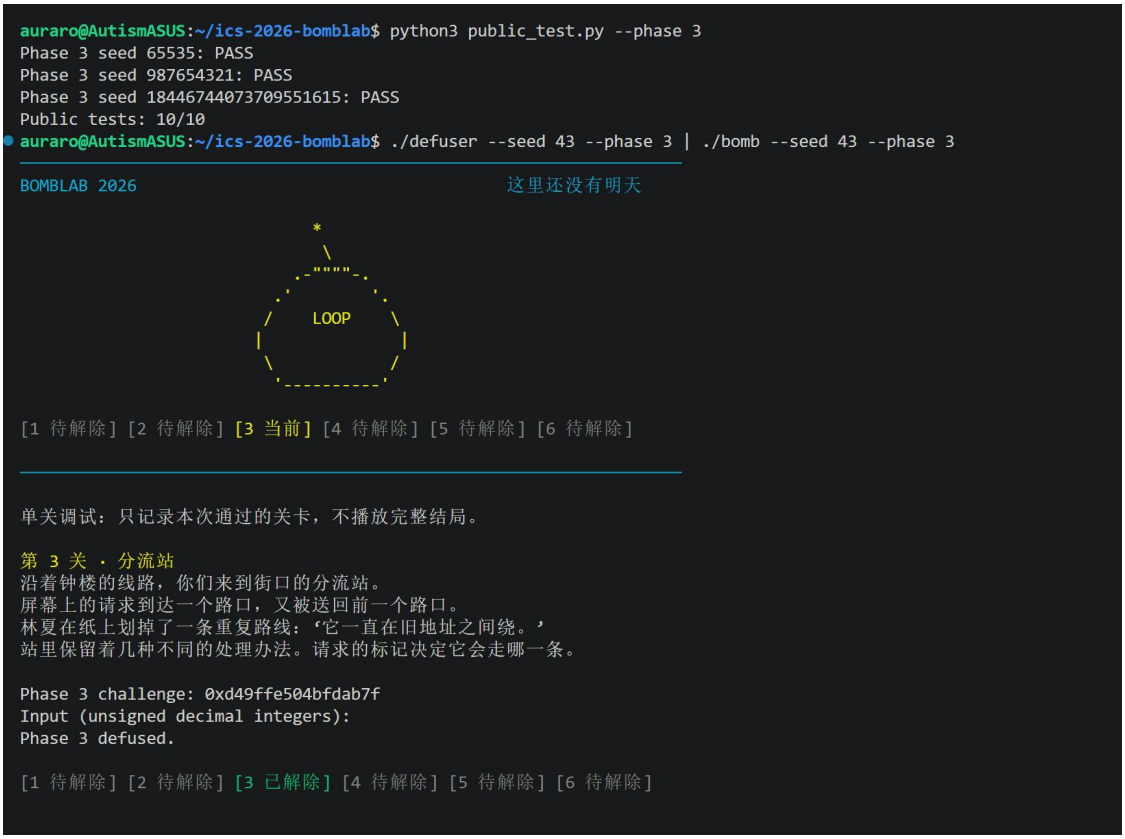


图 3 Phase 3 使用额外 seed 43 的运行结果

五、Phase 4：递归二分搜索与回溯

5.1 递归结构

Phase 4 的辅助函数 p4_walk 接收 target、low、high 和 salt。函数先计算区间中点，再根据 target 与 mid 的大小判断是进入左半区或右半区；相等时会返回基础值。递归返回后，父层还会把子结果乘 3，再与本层的信息异或。

关键汇编

```
mov    %edx,%edi
sub    %esi,%edi
shr    $0x1,%edi
add    %esi,%edi        # mid=low+((high-low) >> 1)


call   p4_walk
lea    (%rax,%rax,2),%eax
xor    %ebx,%eax
xor    $0x1c,%eax       # 右分支；左分支常量为 1


lea    0x87(%rdi),%eax
xor    %ecx,%eax        # 命中 mid 时的基础值
```

外层把 challenge 的低 7 位作为 target，因此范围是 0～127；salt 由右移 7 位的低 32 位与高 32 位异或得到。递归关系可写成下面三种情况。

$$F(t,l,h,s): m = l + ((h-l) \gg 1)$$

$$t = m: F = (m + 0\text{x}87) \oplus s$$

$$t < m: F = 3F(t,l,m-1,s) \oplus (m+s) \oplus 1$$

$$t > m: F = 3F(t,m+1,h,s) \oplus (m+s) \oplus 0\text{x}1\text{c}$$

5.2 用显式栈模拟回溯

为了不额外编写递归辅助函数，我用 pending 数组记录下降过程中每一层的异或量。找到 target 后先生成基础值，再从数组末尾向前计算，顺序正好对应递归的逐层返回。区间最多进行 7 次分割，长度为 8 的数组足够保存整条路径。

对应的 C 代码

```
for (;;) {
```

```
    const uint32_t mid = low + ((high - low) >> 1);
    if (target == mid) {
        answer = (mid + UINT32_C(0x87)) ^ salt;
        break;
    }
    if (target < mid) {
        pending[depth++] = (mid + salt) ^ UINT32_C(1);
        high = mid - 1;
    } else {
        pending[depth++] = (mid + salt) ^ UINT32_C(0x1c);
        low = mid + 1;
    }
}
while (depth > 0)
    answer = (answer * UINT32_C(3)) ^ pending[--depth];
```

seed 43 的 challenge 为 0x006ff81b72d591ff，target=127。二分路径依次经过 63、95、111、119、123、125、126、127，最终答案为 3949747520。完成这一关后，我才真正区分了递归中“向下选择区间”和“向上组合返回值”两个阶段。



图 4 Phase 4 使用额外 seed 43 的运行结果

六、Phase 5：从内存偏移恢复链表

6.1 节点布局

Phase 5 的间接访问集中在三个偏移：(%rdx)、0x8(%rdx)和 0xc(%rdx)。第一个位置被重新装入%rdx，表现得像 next 指针；后两个位置参与 32 位运算，分别可解释为 id 和 bias。由此可以恢复出 16 字节的节点结构。

恢复出的结构体

```
struct Phase5Node {
    const struct Phase5Node *next;  /* 偏移 0 */
    uint32_t id;                    /* 偏移 8 */
    uint32_t bias;                  /* 偏移 12 */
};
```

关键汇编

```
and    $0x7,%eax
mov    0x405b60(,%rax,8),%rdx  # 从 heads 中选择表头
mov    0x8(%rdx),%esi          # id
add    0xc(%rdx),%eax          # acc += bias
lea    (%rax,%rax,4),%eax      # acc *= 5
xor    %esi,%eax               # acc ^= id
mov    (%rdx),%rdx             # node = node->next
```

表头由 challenge&7 选择，遍历次数为((challenge >> 3)&3)+1，所以一定在 1～4 之间。初始累计值由 challenge 的两个 32 位部分组合而来。每访问一个节点就更新累计值，并把当前 id 留作第二个输出。

$$\mathrm{acc}_0 = (\mathtt{uint32_t})(C \gg 6) \oplus (\mathtt{uint32_t})(C \gg 32) \oplus \mathtt{0x6b58}$$

$$\mathrm{acc}_{i+1} = 5(\mathrm{acc}_i + \mathrm{bias}) \oplus \mathrm{id}$$

6.2 seed 43 的链路核对

seed 43 时，challenge 为 0x18e47d0995aecbfe，selector=6，steps=4，选中的链路是节点 9→4→1→7。把每一步的 id、bias 和累计值列出来，可以同时检查静态数据是否抄错、next 指针是否连对。

步骤	节点	id	bias	更新后的 acc
初始	—	—	—	1051897214
1	9	2687	12010	964576375
2	4	2107	42962	528127318
3	1	546	38200	2640828132
4	7	1840	31628	319396608

因此两个输出为“319396608 1840”。第二个数取自最后访问的节点；遍历结束后的next 指针不参与输出。



图 5 Phase 5 使用额外 seed 43 的运行结果

七、Phase 6：函数指针表与两段处理流水线

7.1 两次间接调用

Phase 6 中出现了两次 call *table(...)。第一次用 challenge 的最低 3 位选函数，第二次用位 3～5（最低位编号为 0）选函数。第一次调用后还会异或 0xb483，第二次调用后再加 0x69e9。整个关卡可以看成一条两级流水线。

关键汇编

```
and    $0x38,%ebx
and    $0x7,%eax
call   *0x405e00(,%rax,8)
xor    $0xb483,%eax
...
```

```
call   *0x405e00(%rbx)
add    $0x69e9,%eax
```

第二次索引保留的是 challenge&0x38，它已经等于 selector×8，所以可直接作为字节偏移。读取函数表地址并反汇编每个目标后，得到下表。rol 和 ror 都要显式拼接移出的位，仍按 32 位处理。

selector	运算
0 或 5	$(x + 0\text{x}4294) \oplus (x \gg 2)$
1 或 7	$(x \oplus 0\text{xa}06\text{e}) + 8x$
2	$ror_{32}(x, 5) = (x \gg 5)OR(x \ll 27)$
3	$(4x + 0\text{x}9\text{d}5\text{d}) \oplus 5x$
4	$rol_{32}(x, 3) \oplus 0\text{xa}722$
6	$((x \gg 1) \oplus x) + 0\text{x}3323$

7.2 实现与验证

$$x_0 = (\text{uint32_t})(C \gg 6) \oplus (\text{uint32_t})(C \gg 32)$$

$$\text{answer} = \text{op}_{s_1}(\text{op}_{s_0}(x_0) \oplus 0\text{xb}483) + 0\text{x}69\text{e}9$$

整体控制流程

```
const uint32_t selectors[2] = {
    (uint32_t)(challenge & UINT64_C(7)),
    (uint32_t)((challenge >> 3) & UINT64_C(7))
};
uint32_t x = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);
```

```
for (size_t i = 0; i < 2; ++i) {
    switch (selectors[i]) {
        /* 按函数指针表实现六种运算 */
    }
    if (i == 0) x ^= UINT32_C(0xb483);
}
values[0] = x + UINT32_C(0x69e9);
```

seed 43 的 challenge 为 0x523fb4dd5d162e88，两个 selector 分别为 0 和 1，初始 x=659287143。依次执行 selector 0、异或 0xb483、selector 1，再加 0x69e9，得到答案 2744268288。



图 6 Phase 6 使用额外 seed 43 的运行结果

八、Honor：识别门控字符串并恢复隐藏计算

8.1 secret_gate 的输入布局

Honor 关与前六关最大的区别是输入前面还有一段门控字符串。secret_gate 先检查总长度，再把输入复制到本地缓冲区。随后它在缓冲区偏移 32 的位置比较立即数 0x4e45504f。x86-64 按小端序存放这个数，内存中的四个字节依次是 4f 50 45 4e，也就是字符串“OPEN”。secret_phase 接收的指针又向后移动 36 字节，因此真正的三个十进制整数位于 32 个占位字符和 OPEN 之后。

门控部分关键汇编

```
cmpl   $0x4e45504f,-0xb0(%rbp)  # 偏移 32 处必须为 "OPEN"
jne    <failure>
lea    -0xac(%rbp),%rdi         # 偏移 36，传给 secret_phase
call   secret_phase
```

输入前缀 = 连续 32 个A||OPEN

8.2 三个字段的计算

secret_phase 读取三个 uint32_t。第一个字段 a 等于 context 的最低 2 位。随后把 challenge 和 context 各自的高、低 32 位折叠，再交给 secret_mix。secret_mix 只有一次三倍乘加和一次异或。第二、第三字段都建立在这个结果上。

隐藏计算关键汇编

```
# secret_mix
lea    0x152d(%rdi,%rdi,2),%eax
xor    $0x3409,%eax

# 第三个字段
shr    $0x3,%esi
```

```
lea    0x3409(%r8,%rcx,1),%edx
add    %esi,%edx
xor    %edx,%eax
```

$$a = \text{context\&3}$$

$$c_{\text{fold}} = (\texttt{uint32_t})\text{challenge} \oplus (\texttt{uint32_t})(\text{challenge} \gg 32)$$

$$x_{\text{fold}} = (\texttt{uint32_t})\text{context} \oplus (\texttt{uint32_t})(\text{context} \gg 32)$$

$$\text{mix} = ((c_{\text{fold}} \oplus x_{\text{fold}}) \cdot 3 + \texttt{0x152d}) \oplus \texttt{0x3409}$$

$$b = \text{mix} \oplus ((a \cdot \texttt{0x152d}) \oplus (\texttt{uint32_t})(\text{context} \gg 16))$$

$$c = b \oplus (x_{\text{fold}} + a + \texttt{0x3409} + (c_{\text{fold}} \gg 3))$$

输出格式

```
int size = snprintf(out, cap,
    "AAAAAAAAAAAAAAAA" "AAAAAAAAAAAAAAAA"
    "OPEN%" PRIu32 " %" PRIu32 " %" PRIu32,
    a, b, c);
```

seed 43 时，Honor 输出前缀后的三个数为 3、2567211932、3939006588。bomb 显示“Secret phase defused.”与“[SAFE]”后继续播放隐藏故事，说明门控字符串、context 的使用和三个字段都已匹配。

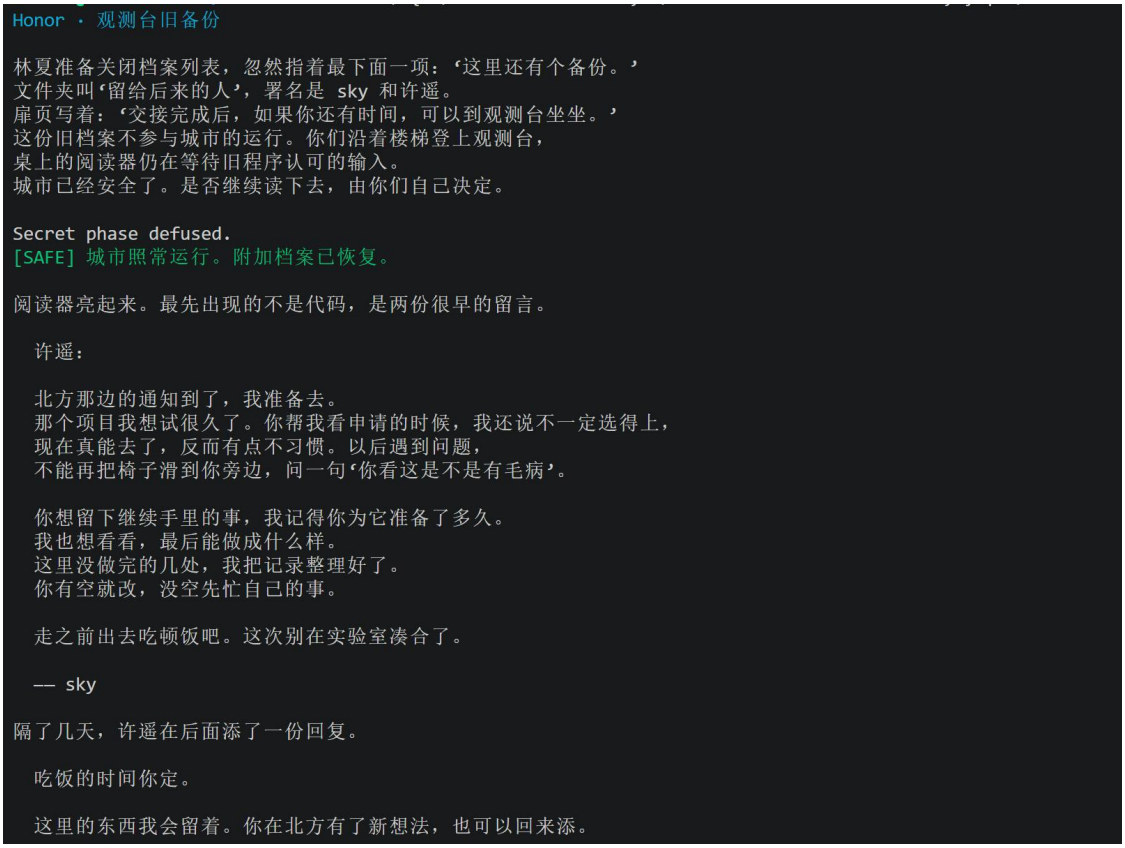


图 7 Honor 关使用额外 seed 43 的解除结果

九、测试结果与完整性检查

9.1 测试汇总

完成每一关后，我先运行 make，确保在-Wall -Wextra -Wpedantic -Werror 下无编译警告，再进行单关测试。全部完成后运行 python3 public_test.py 和 python3 public_test.py --secret，并用 seed 43 把六关答案与 Honor 答案连续送入 bomb。结果如下。

检查项目	结果	说明
普通关公开测试	60/60	Phase 1～6，每关 10 个公开 seed
Honor 公开测试	10/10	10 个公开 seed 全部通过
额外 seed 43	通过	六个普通关卡与 Honor 连续解除
编译检查	通过	开启 -Werror，无警告

检查项目	结果	说明
git diff --check	通过	无空白字符错误

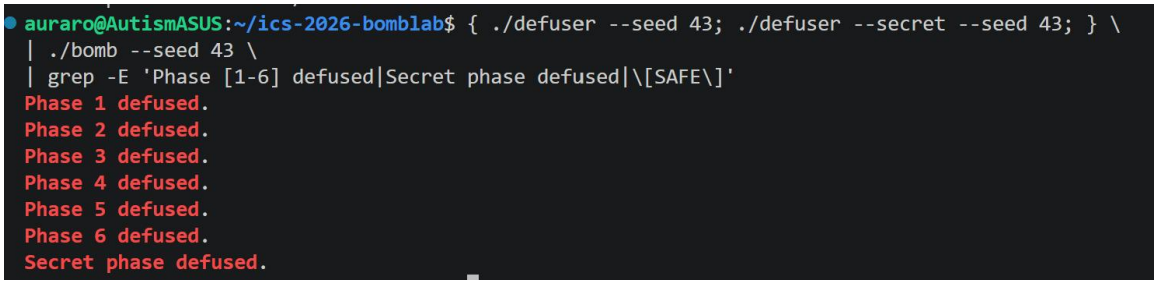


图 8 seed 43 下六个普通关卡和 Honor 关连续通过

9.2 对结果的理解

公开测试全部通过说明公式在多组 challenge 上成立，seed 43 的单独验证则满足了实验对额外 seed 的要求。此外，代码没有保存任何固定 seed 的答案：每一个输出都由当前 challenge（Honor 还包括 context）实时计算，因此面对未公开 seed 时仍能工作。

十、收获与思考

这次实验最明显的变化，是我开始把汇编看成“数据从哪里来、经过什么运算、最后在哪里比较”的过程。坦白来讲，一开始面对整段反汇编时，我完全不知道该怎么入手，只能试着逐行翻译，信息很快就混在一起。后来我先圈出 sscanf、cmp、call 和间接跳转，再给寄存器改成 selector、salt、node、acc 这类名字，控制流就清楚了很多。

六关的难点各不相同。Phase 1 和 Phase 2 训练了位切片、lea 乘加和循环变量；Phase 3 让我意识到函数名不可靠，跳转表中的地址顺序才是实际分支；Phase 4 的重点是递归下降与回溯的先后关系；Phase 5 要从偏移恢复结构体，并沿真实指针连接节点；Phase 6 则把间接 call 还原成函数表。Honor 关把字符串布局、小端序、两个 64 位输入和 32 位运算放到一起，像是对前面方法的一次综合检查。

我还体会到动态调试和静态分析各有合适的位置。objdump 很适合先建立整体结构，GDB 能在某个 cmp 或 call 前后验证猜测，但单次寄存器值不能代替通用公式。把公式写回 C 后再用多个 seed 测试，才算真正完成了从观察到解释的闭环。

最后，我觉得这个实验的故事很有意思，并且与解题过程结合得很巧妙。我们面对的程序只有汇编指令、寄存器和一串看似没有意义的数字，需要一点点还原它真正执行的逻辑；每解开一关，既是在恢复程序的含义，也像是在找回一段被隐藏的记忆，这让原本比较枯燥的反汇编过程有了更强的连贯感。故事最后，sky 写下“谢谢你远道而来，和我相遇”，让我觉得前面的寻找都有了回应。林夏沿着记录走到这里，我们也沿着汇编指令一点点恢复隐藏的内容，最终读到了这句跨越时间的留言。

十一、工具使用说明与参考资料

11.1 工具使用说明

objdump：用于反汇编 phase 函数、辅助函数、跳转表和函数指针表附近代码。GDB：用于设置断点、检查参数寄存器、局部变量和关键指令执行后的结果。make 与 public_test.py：用于编译、运行公开测试和回归检查。

Coding Agent（Codex）：按照课程 Coding Agent 指南使用，主要用于查询个别汇编指令的含义、检查 32 位运算是否存在疏漏、定位代码中的编译错误，以及核对不同 seed 下的测试结果。各关的汇编分析、控制流程还原、公式推导、代码设计与实现均由我独立完成；工具只在完成关键步骤后提供辅助检查。

11.2 参考资料

[1] 复旦大学计算机系统基础课程组. Lab2: BombLab 实验文档. https://ics-26fall-fdu.github.io/labs/lab2-bomb-lab/

[2] ICS-26Fall-FDU. BombLab 模板仓库与 README. https://github.com/ICS-26Fall-FDU/BombLab

[3] 复旦大学计算机系统基础课程组. Coding Agent 使用教程. https://ics-26fall-fdu.github.io/misc/coding-agent/